

Федеральное государственное автономное образовательное учреждение высшего
образования

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет информационных технологий

Кафедра «Информатика и информационные технологии»

Направление подготовки/ специальность: Информационные системы и технологии

ОТЧЕТ

по проектной практике

Студент: Бородина Е.Е., Попова Е.В. Группа: 251-334

Место прохождения практики: Московский Политех, кафедра Управление проектами

Отчет принят с оценкой _____ Дата _____

Руководитель практики: _____

Москва 2026

ОГЛАВЛЕНИЕ

Введение.....	3
1. Общая информация о проекте.....	4
2. Общая характеристика деятельности организации.....	4
3. Описание задания по проектной практике.....	5
3.1 Базовая часть.....	5
3.2 Вариативная часть.....	6
4. Описание достигнутых результатов по проектной практике.....	7
Заключение.....	9
Список литературы.....	10
Приложения.....	11

ВВЕДЕНИЕ

Проектная практика является обязательной частью образовательной программы подготовки студентов направления «Информационные системы и технологии» и направлена на формирование практических навыков разработки программных решений, работы в команде, использования современных инструментов разработки и ведения проектной документации.

В ходе прохождения практики осуществлялась реализация учебного проекта, направленного на закрепление теоретических знаний и развитие компетенций в области программной инженерии, включая использование системы контроля версий Git, языка разметки Markdown, технологий веб-разработки HTML и CSS, а также языка программирования Python для создания Telegram-бота.

Практическая деятельность включала полный цикл разработки программного продукта: от инициализации репозитория и проектирования структуры до реализации функционала, тестирования и документирования результатов.

1. ОБЩАЯ ИНФОРМАЦИЯ О ПРОЕКТЕ

Название проекта

Разработка Telegram-бота с использованием языка программирования Python.

Цель проекта

Целью проекта является разработка программного решения в виде Telegram-бота, а также формирование практических навыков командной разработки, работы с API и инструментами документирования.

Задачи проекта

В рамках реализации проекта были поставлены следующие задачи:

- изучение принципов работы Telegram Bot API;
- настройка среды разработки и инструментов Git;
- разработка архитектуры программного решения;
- реализация базового функционала Telegram-бота;
- обработка пользовательских сообщений;
- тестирование функциональности программного продукта;
- создание веб-сайта проекта;
- оформление технической документации;
- ведение проектного репозитория.

2. ОБЩАЯ ХАРАКТЕРИСТИКА ДЕЯТЕЛЬНОСТИ ОРГАНИЗАЦИИ

Наименование заказчика

Федеральное государственное автономное образовательное учреждение высшего образования «Московский политехнический университет».

Организационная структура

Организационная структура включает следующие элементы:

- руководство образовательной организации;
- факультет информационных технологий;
- выпускающие кафедры;
- руководителей проектной практики;
- кураторов дисциплины «Проектная деятельность».

Данная структура обеспечивает организацию учебного процесса, контроль выполнения проектных работ и методическое сопровождение студентов.

Описание деятельности

Московский политехнический университет осуществляет подготовку специалистов в области информационных технологий, программной инженерии и информационной безопасности.

Особое внимание в образовательном процессе уделяется практико-ориентированному обучению, направленному на развитие у студентов навыков разработки программных систем, командной работы и использования современных инструментов разработки.

Ключевыми направлениями деятельности являются:

- реализация образовательных программ в сфере IT;
- развитие проектного обучения;
- формирование профессиональных компетенций обучающихся;
- внедрение современных технологий разработки программного обеспечения.

3. ОПИСАНИЕ ЗАДАНИЯ ПО ПРОЕКТНОЙ ПРАКТИКЕ

3.1 БАЗОВАЯ ЧАСТЬ

В рамках базовой части были выполнены следующие работы:

- Работа с системой контроля версий Git
- создание и инициализация удалённого репозитория;
- организация структуры проекта;
- выполнение коммитов с осмысленными сообщениями;

- использование ветвления при разработке функциональных компонентов.

Разработка документации

- оформление проектной документации с использованием Markdown;
- описание структуры проекта;
- фиксация этапов разработки;
- ведение журнала изменений.

Разработка веб-сайта проекта

Создан статический веб-сайт с использованием HTML и CSS, включающий:

- главную страницу с описанием проекта;
- страницу «О проекте»;
- раздел «Участники проекта»;
- журнал разработки;
- раздел с ресурсами.

При разработке использовались базовые принципы семантической вёрстки и адаптивного размещения элементов интерфейса.

3.2 ВАРИАТИВНАЯ ЧАСТЬ

Вариативная часть была реализована в виде разработки Telegram-бота на языке Python.

Этапы разработки Telegram-бота

1. Анализ требований. Определение функционального назначения бота и базового набора команд.
2. Проектирование архитектуры. Определена клиент-серверная модель взаимодействия: Пользователь → Telegram API → приложение бота → обработчик команд → ответ пользователю
3. Реализация функционала. Реализованы следующие функции:
 - а. обработка команды /start;

- b. обработка команды /help;
 - c. обработка текстовых сообщений;
 - d. формирование ответных сообщений.
4. Тестирование. Проведена проверка корректности работы всех реализованных функций.
5. Документирование. Подготовлено техническое руководство с описанием установки и использования бота.

Используемые технологии

- Python 3.x
- Telegram Bot API
- библиотека aiogram / python-telegram-bot
- Git для контроля версий

4. ОПИСАНИЕ ДОСТИГНУТЫХ РЕЗУЛЬТАТОВ

В результате выполнения проектной практики были достигнуты следующие результаты:

4.1 Освоение инструментов разработки

Освоены базовые принципы работы с системой контроля версий Git, включая:

- создание репозитория;
- управление изменениями;
- работа с ветками;
- фиксация изменений.

Также освоены основы оформления технической документации с использованием Markdown.

4.2 Реализация Telegram-бота

Разработан Telegram-бот, реализующий базовую логику взаимодействия с пользователем.

Функциональные возможности включают:

- обработку команд;
- обработку текстовых сообщений;
- формирование ответов пользователю.

Архитектура решения построена на событийной модели обработки входящих сообщений.

Исходный код Telegram-бота, техническая документация и материалы проектной практики размещены в удалённом Git-репозитории.

Репозиторий проекта доступен по адресу: *<https://github.com/whotyc/practice>*

4.3 Разработка веб-сайта

Создан статический веб-сайт проекта, содержащий структурированную информацию о проекте, его участниках, этапах разработки и ресурсах.

Сайт разработан с использованием HTML и CSS и предназначен для представления результатов проектной деятельности.

4.4 Итоговая оценка результатов

Все поставленные задачи выполнены в полном объёме. Разработанные компоненты функционируют стабильно и соответствуют требованиям учебного задания.

ЗАКЛЮЧЕНИЕ

В ходе прохождения проектной практики были достигнуты поставленные цели, связанные с формированием практических навыков разработки программных систем и использования современных инструментов программной инженерии.

Выполненные работы включают разработку Telegram-бота на языке Python, создание веб-сайта проекта, ведение проектной документации и использование системы контроля версий Git.

С практической точки зрения проект обладает высокой образовательной ценностью, так как позволяет закрепить навыки работы с API, освоить базовые принципы архитектуры клиент-серверных приложений, а также получить опыт командной разработки и документирования программных решений.

С точки зрения заказчика (образовательной организации) выполненная работа способствует развитию практико-ориентированного обучения и формированию портфолио проектных решений, которые могут использоваться в дальнейшей учебной и методической деятельности.

СПИСОК ЛИТЕРАТУРЫ

1. Python Software Foundation. Python Documentation [Электронный ресурс]. — Режим доступа: <https://docs.python.org/3/> , свободный. — Дата обращения: 12.05.2026.
2. Telegram. Telegram Bot API [Электронный ресурс]. — Режим доступа: <https://core.telegram.org/bots/api> , свободный. — Дата обращения: 12.05.2026.
3. aiogram developers. aiogram Documentation [Электронный ресурс]. — Режим доступа: <https://docs.aiogram.dev/> , свободный. — Дата обращения: 12.05.2026.
4. Chacon S., Straub B. Pro Git [Электронный ресурс]. — Режим доступа: <https://git-scm.com/book/ru/v2> , свободный. — Дата обращения: 12.05.2026.
5. Markdown Guide. Basic Syntax [Электронный ресурс]. — Режим доступа: <https://www.markdownguide.org/basic-syntax/> , свободный. — Дата обращения: 12.05.2026.
6. Mozilla Developer Network. HTML: HyperText Markup Language [Электронный ресурс]. — Режим доступа: <https://developer.mozilla.org/ru/docs/Web/HTML> , свободный. — Дата обращения: 12.05.2026.
7. Mozilla Developer Network. CSS: Cascading Style Sheets [Электронный ресурс]. — Режим доступа: <https://developer.mozilla.org/ru/docs/Web/CSS> , свободный. — Дата обращения: 12.05.2026.
8. Репозиторий проекта MosPolyStudyHelpBot на GitHub [Электронный ресурс]. — Режим доступа: <https://github.com/whotyc/practice> , свободный. — Дата обращения: 15.05.2026.

ПРИЛОЖЕНИЯ

Приложение 1. Листинг кода Telegram-bot

```
import asyncio
from aiogram import Bot, Dispatcher, F
from aiogram.filters import Command
from aiogram.types import Message, CallbackQuery

from config import BOT_TOKEN
from db import (
    init_db,
    add_task, get_tasks, mark_task_done, delete_task, get_stats,
    add_note, get_notes, get_note_by_user_num, get_note_by_id, search_notes,
    delete_note,
    add_lesson, get_schedule, delete_lesson, get_all_lessons,
    get_deadline_tasks
)
from keyboards import main_menu, tasks_menu, notes_menu, schedule_menu,
weekday_inline
from utils import parse_deadline, format_deadline, days_until
from datetime import datetime, timedelta

bot = Bot(token=BOT_TOKEN)
dp = Dispatcher()

user_state = {}

async def reminder_loop():
    while True:
        try:
            await asyncio.sleep(60)
        except Exception:
            pass

@dp.message(Command("start"))
async def cmd_start(message: Message):
    await message.answer(
        "Привет! Я StudyHelperBot 📖\n\n"
        "Я помогу тебе:\n"
        "✎ вести задачи и дедлайны\n"
        "💬 хранить заметки\n"
        "📅 вести расписание\n"
        "📊 смотреть статистику\n\n"
        "Выбирай раздел в меню 👇",
        reply_markup=main_menu()
    )

@dp.message(Command("help"))
async def cmd_help(message: Message):
    await message.answer(
        "❓ Команды бота:\n\n"
        "/start – запуск\n"
        "/help – помощь\n\n"
        "Все функции доступны через кнопки меню."
    )

@dp.message(F.text == "❓ Помощь")
```

```

async def help_btn(message: Message):
    await cmd_help(message)

@dp.message(F.text == "📌 Задачи")
async def tasks_section(message: Message):
    await message.answer("Раздел задач 📌", reply_markup=tasks_menu())

@dp.message(F.text == "📝 Заметки")
async def notes_section(message: Message):
    await message.answer("Раздел заметок 📝", reply_markup=notes_menu())

@dp.message(F.text == "📅 Расписание")
async def schedule_section(message: Message):
    await message.answer("Раздел расписания 📅", reply_markup=schedule_menu())

@dp.message(F.text == "⬅️ Назад")
async def back_to_main(message: Message):
    user_state.pop(message.from_user.id, None)
    await message.answer("Главное меню:", reply_markup=main_menu())

# -----
# Tasks
# -----

@dp.message(F.text == "+ Добавить задачу")
async def task_add_start(message: Message):
    user_state[message.from_user.id] = {"mode": "add_task_title"}
    await message.answer("Введите название задачи:")

@dp.message(F.text == "📋 Мои задачи")
async def task_list(message: Message):
    tasks = await get_tasks(message.from_user.id, only_active=True)

    if not tasks:
        await message.answer("Активных задач нет ☑️")
        return

    text = "📋 Твои активные задачи:\n\n"
    for num, title, deadline, done in tasks:
        text += f"#{num} - {title} (🕒 {format_deadline(deadline)})\n"

    await message.answer(text)

@dp.message(F.text == "☑️ Выполнить задачу")
async def task_done_start(message: Message):
    user_state[message.from_user.id] = {"mode": "done_task"}
    await message.answer("Введите номер задачи (например: 3):")

@dp.message(F.text == "🗑️ Удалить задачу")
async def task_delete_start(message: Message):
    user_state[message.from_user.id] = {"mode": "delete_task"}
    await message.answer("Введите номер задачи для удаления:")

# -----

```

```

# Notes
# -----

@dp.message(F.text == "➕ Добавить заметку")
async def note_add_start(message: Message):
    user_state[message.from_user.id] = {"mode": "add_note_title"}
    await message.answer("Введите заголовок заметки:")

@dp.message(F.text == "📖 Мои заметки")
async def notes_list(message: Message):
    notes = await get_notes(message.from_user.id)
    if not notes:
        await message.answer("Заметок пока нет 🙄")
        return

    text = "📖 Твои заметки:\n\n"
    for num, title, created_at in notes:
        text += f"#{num} - {title}\n"

    text += "\nЧтобы прочитать заметку, напиши: /note <номер>"

    await message.answer(text)

@dp.message(Command("note"))
async def read_note(message: Message):
    parts = message.text.split()
    if len(parts) != 2 or not parts[1].isdigit():
        await message.answer("Используй формат: /note 3")
        return

    user_num = int(parts[1])
    note = await get_note_by_user_num(message.from_user.id, user_num)

    if not note:
        await message.answer("Заметка не найдена.")
        return

    title, content, created_at = note
    await message.answer(f"📖 {title}\n\n{content}")

@dp.message(F.text == "🔍 Поиск заметки")
async def note_search_start(message: Message):
    user_state[message.from_user.id] = {"mode": "search_note"}
    await message.answer("Введите слово или фразу для поиска:")

@dp.message(F.text == "🗑 Удалить заметку")
async def note_delete_start(message: Message):
    user_state[message.from_user.id] = {"mode": "delete_note"}
    await message.answer("Введите номер заметки для удаления:")

# -----
# Schedule
# -----

@dp.message(F.text == "➕ Добавить пару")
async def schedule_add_start(message: Message):
    user_state[message.from_user.id] = {"mode": "add_lesson_day"}
    await message.answer("Выберите день недели:", reply_markup=weekdays_inline())

```

```

@dp.callback_query(F.data.startswith("day:"))
async def choose_day(callback: CallbackQuery):
    user_id = callback.from_user.id
    if user_id not in user_state or user_state[user_id].get("mode") !=
"add_lesson_day":
        await callback.answer()
        return

    day = int(callback.data.split(":")[1])
    user_state[user_id] = {"mode": "add_lesson_time", "day": day}

    await callback.message.answer("Введите время пары (например 09:30):")
    await callback.answer()

@dp.message(F.text == "📅 Показать расписание")
async def show_schedule(message: Message):
    lessons = await get_all_lessons(message.from_user.id)

    if not lessons:
        await message.answer("Расписание пустое 📅")
        return

    days = {
        1: "Понедельник", 2: "Вторник", 3: "Среда",
        4: "Четверг", 5: "Пятница", 6: "Суббота", 7: "Воскресенье"
    }

    text = "📅 Твоё расписание:\n\n"
    current_day = None

    for num, weekday, time, subject in lessons:
        if weekday != current_day:
            current_day = weekday
            text += f"\n📅 {days[weekday]}:\n"
        text += f"#{num} ☹ {time} - {subject}\n"

    await message.answer(text)

@dp.message(F.text == "🗑 Удалить пару")
async def delete_lesson_start(message: Message):
    user_state[message.from_user.id] = {"mode": "delete_lesson"}
    await message.answer(
        "Введите номер пары для удаления.\n"
        "Номера видны в списке расписания (📅 Показать расписание)."
    )

@dp.message(F.text == "📊 Статистика")
async def stats(message: Message):
    total, done = await get_stats(message.from_user.id)
    active = total - done

    await message.answer(
        f"📊 Статистика:\n\n"
        f"Всего задач: {total}\n"
        f"Выполнено: {done}\n"
        f"Активно: {active}"
    )

# -----

```

```

# Universal text handler (FSM via user_state)
# -----

@dp.message()
async def handle_text(message: Message):
    user_id = message.from_user.id
    if user_id not in user_state:
        return

    state = user_state[user_id]
    mode = state.get("mode")

    # ---- TASK ADD ----
    if mode == "add_task_title":
        title = message.text.strip()
        user_state[user_id] = {"mode": "add_task_deadline", "title": title}
        await message.answer(
            "Введите дедлайн в формате:\n"
            "YYYY-MM-DD или YYYY-MM-DD HH:MM\n\n"
            "Если дедлайн не нужен – напишите: нет"
        )
        return

    if mode == "add_task_deadline":
        title = state["title"]
        text = message.text.strip()

        deadline = None
        if text.lower() != "нет":
            dt = parse_deadline(text)
            if not dt:
                await message.answer("Неверный формат. Попробуйте снова.")
                return
            deadline = dt.isoformat()

        await add_task(user_id, title, deadline)
        user_state.pop(user_id, None)
        await message.answer("☑ Задача добавлена!", reply_markup=tasks_menu())
        return

    if mode == "done_task":
        if not message.text.isdigit():
            await message.answer("Введите число (номер задачи).")
            return

        user_num = int(message.text)
        found = await mark_task_done(user_id, user_num)
        user_state.pop(user_id, None)
        if found:
            await message.answer("☑ Задача отмечена как выполненная.",
reply_markup=tasks_menu())
        else:
            await message.answer("Задача с таким номером не найдена.",
reply_markup=tasks_menu())
        return

    if mode == "delete_task":
        if not message.text.isdigit():
            await message.answer("Введите число (номер задачи).")
            return

        user_num = int(message.text)
        found = await delete_task(user_id, user_num)

```

```

        user_state.pop(user_id, None)
        if found:
            await message.answer("🗑️ Задача удалена.", reply_markup=tasks_menu())
        else:
            await message.answer("Задача с таким номером не найдена.",
reply_markup=tasks_menu())
            return

# ---- NOTE ADD ----
if mode == "add_note_title":
    title = message.text.strip()
    user_state[user_id] = {"mode": "add_note_content", "title": title}
    await message.answer("Введите текст заметки:")
    return

if mode == "add_note_content":
    title = state["title"]
    content = message.text.strip()

    await add_note(user_id, title, content)
    user_state.pop(user_id, None)
    await message.answer("✅ Заметка сохранена!", reply_markup=notes_menu())
    return

if mode == "search_note":
    query = message.text.strip()
    results = await search_notes(user_id, query)

    if not results:
        await message.answer("Ничего не найдено.")
    else:
        text = "🔍 Найдено:\n\n"
        for num, title in results:
            text += f"#{num} - {title}\n"
        text += "\nЧтобы открыть: /note <номер>"
        await message.answer(text)

    user_state.pop(user_id, None)
    return

if mode == "delete_note":
    if not message.text.isdigit():
        await message.answer("Введите число (номер заметки).")
        return

    user_num = int(message.text)
    found = await delete_note(user_id, user_num)
    user_state.pop(user_id, None)
    if found:
        await message.answer("🗑️ Заметка удалена.", reply_markup=notes_menu())
    else:
        await message.answer("Заметка с таким номером не найдена.",
reply_markup=notes_menu())
    return

# ---- SCHEDULE ADD ----
if mode == "add_lesson_time":
    time = message.text.strip()
    if len(time) != 5 or time[2] != ":":
        await message.answer("Введите время в формате HH:MM (например 09:30).")
        return

    user_state[user_id]["time"] = time
    user_state[user_id]["mode"] = "add_lesson_subject"

```



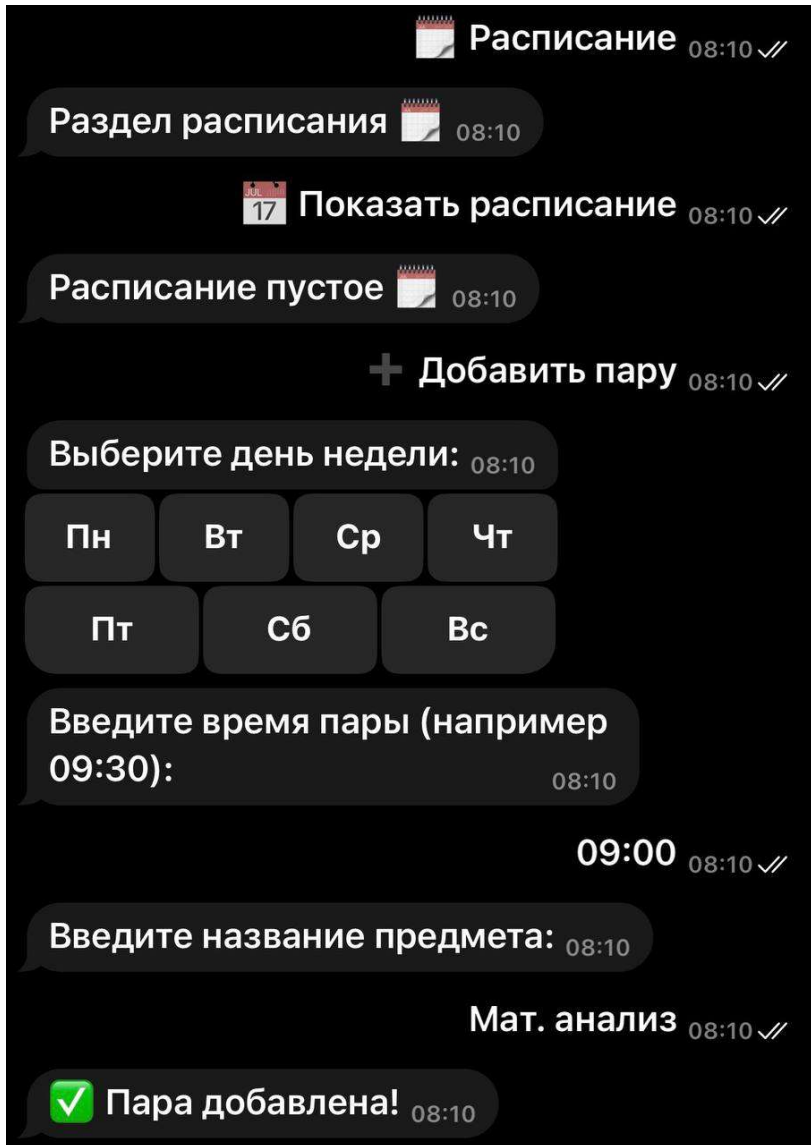
```

    await message.answer("Введите название предмета:")
    return

if mode == "add_lesson_subject":
    subject = message.text.strip()
    day = state["day"]
    time = state["time"]

```

Приложение 2. Пример взаимодействия с ботом



Приложение 3. Листинг кода сайта

```

<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>Steamleaf – Главная</title>
  <link rel="stylesheet" href="css/style.css"/>
  <link rel="icon" type="image/png" href="images/logo.png"/>
</head>
<body>

<header class="header">

```

```

<div class="container nav">
  <a class="logo" href="index.html">
    
    <span>Steamleaf</span>
  </a>
  <nav class="menu">
    <a class="active" href="index.html">Главная</a>
    <a href="about.html">О6 игре</a>
    <a href="team.html">Команда</a>
    <a href="journal.html">Дневник</a>
    <a href="resources.html">Ресурсы</a>
    <a href="bot.html">Бот</a>
  </nav>
</div>
</header>

<section class="hero">
  <div class="container hero-grid">
    <div class="hero-text">
      <h1>Симулятор раздачи <em>листовок</em></h1>
      <p class="subtitle">
        Steamleaf – инди-игра, в которой вы играете за уличного промютера в
        туманном городе.
        Раздавайте листовки, убеждайте прохожих, зарабатывайте репутацию – и,
        возможно, изменитесь сами.
      </p>
      <div class="hero-buttons">
        <a class="btn primary" href="about.html">О6 игре</a>
        <a class="btn secondary" href="journal.html">Дневник разработки</a>
      </div>
      <div class="hero-badges">
        <span class="badge">Unity</span>
        <span class="badge">C#</span>
        <span class="badge">2D</span>
        <span class="badge">HTML/CSS</span>
        <span class="badge">Git</span>
      </div>
    </div>

    <div class="hero-card">
      <h3>Механики игры</h3>
      <ul>
        <li>📄 Раздача листовок прохожим</li>
        <li>💰 Система денег и репутации</li>
        <li>🌙 Смена дня и ночи</li>
        <li>💬 Взаимодействие с NPC и задания</li>
        <li>🏠 Квартира и открытый мир</li>
      </ul>
      <p class="small-note">Учебный игровой проект, разработан в рамках практики,
      2026.</p>
    </div>
  </div>
</section>

<section class="section">
  <div class="container">
    <h2>Что такое Steamleaf?</h2>
    <div class="cards">
      <div class="card">
        <h3>📄 Листовки и город</h3>
        <p>Вы выходите на улицы Стимлифа с пачкой листовок. Каждый прохожий –
        отдельная история: кто-то возьмёт сам, кто-то убежит.</p>
      </div>
      <div class="card">

```

```

        <h3>🌍 Живой мир</h3>
        <p>Туманный город меняется со временем суток. NPC живут по своему
расписанию, дают задания и реагируют на вашу репутацию.</p>
    </div>
    <div class="card">
        <h3>📊 Прогресс</h3>
        <p>Деньги и репутация определяют, что вам доступно. Выполняйте задания
горожан, чтобы открыть новые возможности.</p>
    </div>
</div>
</div>
</section>

<section class="section section-alt">
    <div class="container">
        <h2>Технологии проекта</h2>
        <div class="cards">
            <div class="card">
                <h3>⚙️ Движок</h3>
                <p>Unity + C#. Вся игровая логика, AI прохожих, системы денег и репутации
написаны на C#.</p>
            </div>
            <div class="card">
                <h3>🎨 Графика</h3>
                <p>2D-спрайты и тайловые карты, собранные в Unity Tilemap. Дизайн персонажей
– авторские наброски.</p>
            </div>
            <div class="card">
                <h3>📖 Документация</h3>
                <p>Markdown, Git, дневник разработки и руководство по запуску проекта.</p>
            </div>
        </div>
    </div>
</section>

<footer class="footer">
    <div class="container footer-grid">
        <div>
            <a class="logo" href="index.html">
                
                <span>Steamleaf</span>
            </a>
        </div>
        <div>
            <h4>Навигация</h4>
            <a href="index.html">Главная</a>
            <a href="about.html">Об игре</a>
            <a href="team.html">Команда</a>
            <a href="journal.html">Дневник</a>
            <a href="resources.html">Ресурсы</a>
            <a href="bot.html">Бот</a>
        </div>
        <div>
            <h4>Авторы</h4>
            <p>Попова Е.В.</p>
            <p>Бородина Е.Е.</p>
        </div>
    </div>
</footer>

</body>
</html>

```

Приложение 4. Главная страница сайта

